programming language) programmer learning JavaScript (a domain-specific programming language). The programming experience might also depend on the underlying architecture and operating system. An x86 assembly language programmer (from Intel) learning MIPS assembly language (from MIPS Technologies) or a PowerShell (from Microsoft) user learning Linux shell scripting are bound to encounter a few surprises. Of course, there could be many other reasons for this, but the above mentioned ones seem to be the most obvious.

For those learning their first programming language, it's most probable that none of the features are surprising. Now let us look at a few features of programming languages that might surprise someone who learns these as their second programming language.

Complicated pointers in C/C++

Most undergraduate programmes in computer science do have a course on C programming and, to many, the most difficult section is the one on pointers. To add to the misery, you can declare pointers of arbitrary complexity. As an example, consider the C program named *pointer.c* given below. This and all the other programs discussed in this article can be downloaded from *opensourceforu.com/article_source_code/June20surprisinglanguage.zip.*

```
#include<Stdio.h>

int main()
{
    int *(*(* ptr)(int *))[2];
    int *******ptr1;
    int ******************************
******************ptr3;
    return 0;
}
```

The above C program compiles without any errors. The pointer variable *ptr1* is a pointer to a function that accepts a pointer to an integer as an argument and returns a pointer to an array of pointers to integers. The pointer variable *ptr2* is a pointer to a pointer to a pointer to a pointer to a pointer to a pointer to a pointer to an integer (I hope I have counted correctly!). There are 50 asterisk symbols before the declaration of the pointer variable *ptr3.* But is there a limit to this? I tried up to 1000 asterisk symbols and it was still compiling fine. I believe there are no upper limits set by the C standard. There may be a limit at which the C compiler might fail to handle this, but I don't know where that limit is. No programmer will ever use these sorts of pointers in a real program. Nevertheless, these features are available for us to use. A similar C++ program will also compile without any errors. Now back to our business. Imagine the horror of a Java, Python or Haskell (all of which are programming languages that do not use pointers) programmer who comes across such monstrosities.

Confusing Octal numbers in C/C++/Java

This may not be as big a surprise as the previous one. But once, a long time back, this feature of C/C++/Java gave me quite a headache and I believe we should discuss it. Consider the C++ program *octal.cc* given below. Similar C and Java programs (*octal.c* and *Octal.java)* are also available for download. What is the output of the program *octal.cc*?

```
#include<iostream>
using namespace std;

int main()
{
    int num[ ]={001,005,025,125,625};
    for(int i=0;i<5;i++)
    {
        cout<<num[i]<<endl;
    }
    return 0;
}
```

The array *num* contains the first five numbers in the geometric sequence starting at 1 with common ratio 5. Figure 1 shows the output of the program *octal.cc.* But why is the number 21 printed instead of 25? Well, in C, C++ and Java, a number like 0123 is treated as an octal number and '0x123' is treated as a hexadecimal number. In the program *octal.cc* the numbers 001, 005 and 025 are treated as octal numbers due to this reason. But the numbers 001 and 005 are the same in decimal and octal number systems, whereas the number 025 in octal is 21 in decimal. Thus, the sequence printed is 1, 5, 21, 125, 625. This feature is convenient on many occasions, but could lead to potential bugs if one is not careful. For example, a Python programmer who is familiar with the notation '0o25' might consider '025' as the number 25 with a leading zero.

```
# g++ octal.cc
# ./a.out
1
5
21
125
625
#
```

Figure 1: Output of the C++ program *octal.cc*

The *for* loop with an *else* in Python

As mentioned earlier, a C/C++/Java programmer newly learning Python will be surprised that explicit type declaration is not required in Python. Since this dynamic typing feature of Python is well known to many programmers, I will discuss a simple yet surprising feature of Python; *for* loop with an *else* part. Consider the Python script *loop.py* shown below.

```
for i in range(5):
    print(i)
else:
    print("Noraml Exit from for loop")
for i in range(10):
```